Data Dictionary

Table of Contents

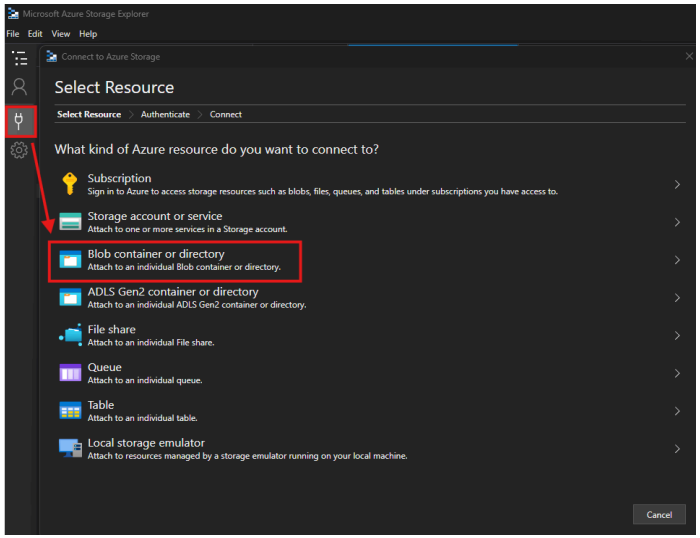
Accessing the Blob Containers on Microsoft Azure Storage Explorer

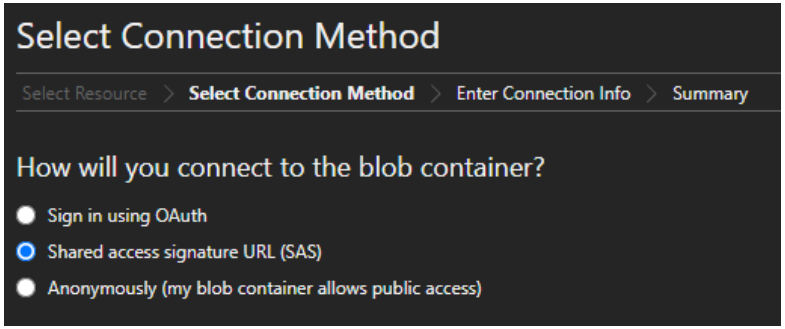
[CheXpert: Chest X-Rays]
To use the CheXpert files, request access using this link:
https://stanfordaimi.azurewebsites.net/datasets/8cbd9ed4-2eb9-4565-affc-111cf4f7ebe2

[CheXlocalize]
To access the test radiologist-labeled datasets, request access using this link:
https://stanfordaimi.azurewebsites.net/datasets/23c56a0d-15de-405b-87c8-99c30138950c

For each, a link to the blob will be provided which can be accessed using some Microsoft Azure application. We used Microsoft Azure Storage Explorer which provides a useful UI for navigation purposes.

Open the connect dialog box, then select “Blob container or directory.” Then, select “Shared access signature URL (SAS)”





On the following page, paste the link to the blob in the “Blob container or directory SAS URL” field, and the “Display name” field will auto populate. Then, click Next. If everything was done correctly, the files should appear in the Storage Explorer UI. Only the below files and folders are necessary for this project

Explanation of Files From the CheXpert Blob

Name	Content Type	Size	Last Modified	Parent Directory	Description	Required
CheXpert-v1.0 batch 1 (validate & csv).zip	Compressed Zip Folder	486.04 MB	12/30/2023	-	Contains the **radiologist-labeled validation** images (in /valid/) as well as associated metadata in valid.csv	**YES ****
valid	Compressed Zip Folder	484 MB	10/19/2018	CheXpert-v1.0 batch 1 (validate & csv).zip	Contains the **radiologist-labeled validation** images	**YES ****
valid.csv	CSV	31 KB	1/20/2019	CheXpert-v1.0 batch 1 (validate & csv).zip	Contains the **radiologist-labeled validation** image paths and associated metadata	**YES ****
CheXpert-v1.0 batch 2 (train 1).zip	Compressed Zip Folder	162.39 GB	12/30/2023	-	Contains the first set of training images	**YES**
CheXpert-v1.0 batch 3 (train 2).zip	Compressed Zip Folder	184.82 GB	12/30/2023	-	Contains the second set of training images	**YES**
CheXpert-v1.0 batch 4 (train 3).zip	Compressed Zip Folder	91.09 GB	12/30/2023	-	Contains the third set of training images	**YES**
README.md	Plain-text	3.21 KB	8/9/2021	-	A plain-text file which documents the dataset, the images, and the labeling	**YES ***
train_cheXbert.csv	CSV	22.06 MB	8/9/2021	-	The training labels produced by the CheXbert labeler	**YES**

** The README file is not required to run the project, but it contains useful information about the datasets.*

*** The radiologist-labeled validation files are used to benchmark our model's performance against human-labeling, but are not required to train and run the model.*

Explanation of Files From the CheXlocalize Blob

Name	Content Type	Size	Last Modified	Parent Directory	Description	Required
test	Folder	1.94 MB	8/9/2021	-	Contains limited demographic information for each patient in the dataset	**YES ****
test_labels.csv	CSV	31 KB	1/20/2019	CheXpert-v1.0 batch 1 (validate & csv).zip	Contains the **radiologist-labeled validation** image paths and associated metadata	**YES ****

*** The radiologist-labeled test files are used to benchmark our model's performance against human-labeling, but are not required to train and run the model.*

Starting Schema Configuration

In order for images to be located properly, the source data must be downloaded from the blob and organized into the following directory structure:

| **MAIN_DIR**
|------ | **data**
|------ | **raw_data**
|-----| **CheXpert-v1.0 batch 1 (validate & csv)**
| |----- |-----| **valid**
| |-----| valid.csv
|-----| **CheXpert-v1.0 batch 2 (train 1)**
|-----| **CheXpert-v1.0 batch 3 (train 2)**
|-----| **CheXpert-v1.0 batch 4 (train 3)**
|-----| **test**
|-----| test_labels.csv
|-----| train_cheXbert.csv

Directories
MAIN_DIR: The directory containing all project files
data : The directory which will contain all project data (images and data frames)
raw_data: The directory containing all images and data frames as downloaded from the blobs

Patient directory structure
| Patient ID
|-----| Study ID
|----------| Frontal View / Lateral View

Example
File path: CheXpert/data/raw_data/CheXpert-v1.0 batch 3 (train 2)/patient21521/study3/view1_frontal.jpg

Required Non-Standard Python Libraries

Library Name	Documentation Link	Description
torch	https://pytorch.org/get-started/locally/	This library powers our neural network models
torchvision	https://pytorch.org/get-started/locally/	Torchvision contains the pre-trained ResNet50 and DenseNet121 models for transfer learning
numpy-hilbert-curve	https://github.com/PrincetonLIPS/numpy-hilbert-curve	We use this library during our Grad-CAM analysis to better encode spatial relationships in a 1-D vector
grad-cam	https://github.com/jacobgil/pytorch-grad-cam	Grad-CAM is used to visualize the decision-making of our model
yaml	https://pyyaml.org/wiki/PyYAMLDocumentation	This library is used to store preset configuration parameters.

Data Transformation Steps

If the image files and label data frames are saved according to pages 2-4, then the below actions can be performed using Preprocessing1_EnvironmentSetup.ipynb (Steps A-B) and Preprocessing2_ImagePreprocessing.ipynb (Step C)

Step	Description	Input Signature	Relevant Prior Step	Output Signature	Tool/Library Required
(A) File Organization					
A1	Download, extract, and organize the required files into the required directory structure	-	Reference pages 2-4	-	File Explorer, Microsoft Azure Storage Explorer
A2	Copy the directory structures of the image folders. This is performed in Preprocessing1_EnvironmentSetup	-	A1	-	File Explorer, Preprocessing1_EnvironmentSetup.ipynb
(B) Data Frame Preparation Though the exact processing steps are detailed below, the code to run these functions are contained within Preprocessing1_EnvironmentSetup.ipynb.					
B1	From train_cheXbert.csv, extract the rows with file paths to images contained within CheXpert-v1.0 batch 4 (train 3) as the test set	Data Frame	A2	Data Frame	Pandas
B2	From train_cheXbert.csv, extract the rows with file paths to images contained within CheXpert-v1.0 batch 2 (train 1) and CheXpert-v1.0 batch 3 (train 2) as the combined train_validation set	Data Frame	A2	Data Frame	Pandas
B3	Shuffle the train_validation set	Data Frame	B2	Data Frame	Pandas
B4	Split the train_validation set into a train set and validation set using a 80%-20% ratio	Data Frame	B3	Data Frame	Pandas, sklearn.model_selection.train_test_split()
B5	From valid.csv, extract the rows with file paths to images contained within CheXpert-v1.0 batch 1 (validate & csv) as the radiologist-labeled validation set	Data Frame	A2	Data Frame	Pandas
B6	From test_labels.csv, extract the rows with file paths to images contained within test as the radiologist-labeled validation set	Data Frame	A2	Data Frame	Pandas
B7	Add new columns containing file paths to preprocessed image variations	Data Frame	B1-B6	Data Frame	Pandas
B8	Filter out unnecessary conditions columns (Keep only Cardiomegaly, Consolidation, Edema, Atelectasis, Pleural Effusion)	Data Frame	B7	Data Frame	Pandas

B9	Remove lateral images from the data frames	Data Frame	B8	Data Frame	Pandas
B10	Remove rows where all five condition columns are 0	Data Frame	B9	Data Frame	Pandas
B11	Create train_df_oversampled	Data Frame	B10	Data Frame	Pandas, scripts.prepare.resample()
B12	Sample 30,000 images from the training dataset with weighted replacement	Data Frame	B11	Data Frame	Pandas, scripts.prepare.resample()
B13	Downsample the original training set by removing 30,000 images randomly	Data Frame	B12	Data Frame	Pandas, scripts.prepare.resample()
B14	Concatenate the sample of 30,000 with the downsampled training set	Data Frame	B13	Data Frame	Pandas

(C) Image Preprocessing Pipelines (Performed up-front for both the raw and enhanced images)

Though the exact processing steps are detailed below, these are all wrapped into the **scripts.preprocessing.run_pipeline()** function for convenience. The code to run this function is also contained within Preprocessing2_ImagePreprocessing.ipynb

scripts.preprocessing.run_pipeline(df, dims, skip_if_image_exists=True**, **kwargs)**

Inputs:
df [*Pandas Data Frame*]: Data frame containing the image labels and file paths
dims [*List*]: List of output image dimensions. This must be [224], [384] or [284, 384]. If converting to a single dimension, this should still be input as a list.
skip_if_image_exists [*bool*]: If a file path already leads to an existing image, continue to the next file path
****kwargs** [*dict*], optional: A dictionary containing keyword arguments from the image processing pipeline. By default, these are the settings:
- **usm_weight** = 1.2 [*float*]: The weight value for unsharp masking
- **usm_sigma**= 10 [*float*]: The Gaussian blur parameter for unsharp masking
- **he_sigma**= 5 [*float*]: The Gaussian blur parameter for histogram equalization
- **scale_min**= 0 [*float*]: The output minimum pixel value
- **scale_max**= 255 [*float*]: The output maximum pixel value

Returns:
imgs1 [list]: A list of images that went through the C2-C7 and C11-C12 pipeline
imgs2 [list]: A list of images that went through the complete C2-C12 pipeline

C1	Run the image processing pipeline on a specified data frame. This function will result in two saved copies (one “raw” and one “enhanced”) to the output file path associated to the input dims Below are the step-by-step transformations:	Data Frame	B14	.jpeg images	scripts.preprocessing.run_pipeline()
C2	Load image from the source file path as an integer Tensor	.jpeg file	C1	Int Tensor	torchvision.io.decode_image()

C3	Resize the training/validation/testing images to either 224x224 or 384x384 with bilinear interpolation	Int Tensor	C2	Float Tensor	v2.Resize()
C4	Convert float Tensor to numpy array	Float Tensor	C3	Float array	tensor.numpy()
C5	Rescale the image pixel values to the range [0,255]	Float array	C4	Float array	scripts.preprocessing.scale_range()
C6	Convert float array to uint8 array	Float array	C5	Uint8 array	arr.astype(np.uint8)
C7	Extract the 2nd and 3rd axes (img = img[0,:,:]) to get a 2D representation	Uint8 array	C6	Uint8 array	arr[0, :, :]
C8	(Enhanced Images) Rescale the image pixel values to the range [0,255]	Uint8 array	C7	Float array	scripts.preprocessing.scale_range()
C9	(Enhanced Images) Sharpen the image using unsharp masking	Float array	C8	Float array	scripts.preprocessing.unsharp_masking()
C10	(Enhanced Images) Equalize the histogram to increase contrast	Float array	C9	Float array	scripts.preprocessing. histogram_equalization()
C11	Convert array to type uint8	Float array	C10	Uint8 array	arr.astype(np.uint8)
C12	Convert integer array to Image object	Uint8 array	C11	Image object	Image.fromarray(arr)
C13	Save the processed images as .jpeg files with 0.95 quality	Image object	C12	.jpeg file	Image.save()
(D) Training Image Processing Pipeline (Performed only within the PyTorch dataloader)					
D1	Load Image as a 3D Tensor (RGB instead of Grayscale)	.jpeg file	C13	Int Tensor	torchvision.io.decode_image()
D2	Convert integer tensor to float	Int Tensor	D1	Float Tensor	Tensor.float()
D3	Randomly rotate by a 90-degree interval	Float Tensor	D2	Float Tensor	torch.rot90()
D4	Randomly rotate by a 30 degree interval	Float Tensor	D3	Float Tensor	v2.functional.rotate()
D5	Randomly crop either 0, 16, 32, or 48 pixels from the edge	Float Tensor	D4	Float Tensor	v2.CenterCrop()
D6	If a non-zero number of pixels were cropped, resize back to the original dimensions with bilinear interpolation	Float Tensor	D5	Float Tensor	v2.Resize()
D7	Normalize according to the specified mean and standard deviation values specified here: ResNet: https://pytorch.org/hub/pytorch_vision_resnet/ DenseNet: https://pytorch.org/hub/pytorch_vision_densenet/	Float Tensor	D6	Float Tensor	v2.Normalize()

Explanation of Variables

train_cheXbert.csv contains the file path to each x-ray image along with corresponding features that indicate the presence (or lack thereof) of 14 pathological conditions. It also contains some limited demographic information regarding the patient as well as the configuration parameters of the x-ray itself.

Field Name	Data Type	Field Size	Description	Field Type
Path	TEXT	58	The file path leading to the x-ray image.	ID
Sex	TEXT	7	The gender of the patient.	Demographic
Age	INT64	3	The age of the patient when the study was performed.	Demographic
Frontal/Lateral	TEXT	7	This indicates whether the x-ray view was taken from a frontal position, or a lateral position	X-ray configuration
AP/PA	TEXT	3	X-ray code which indicates the view angle Posteroanterior (PA) refers to when the x-ray beam passes through the patient from the back to the front Anteroposterior (AP) refers to when the x-ray beam passes through the patient from the front to the back Lower Lumbar (LL) refers to views of the lumbar spine and sacrum area RL refers to views taken from either the right (R) or left (L) sides of the body	X-ray configuration
Enlarged Cardiomediastinum	FLOAT64	3	This indicates if the cavity containing the heart and other structures is enlarged.	Pathological condition
Cardiomegaly	FLOAT64	3	This indicates the presence of an enlarged heart.	Pathological condition
Lung Opacity	FLOAT64	3	This indicates the presence of hazy, dense areas in the lung that should be darker.	Pathological condition
Lung Lesion	FLOAT64	3	This indicates an abnormal growth in the lung tissue.	Pathological condition
Edema	FLOAT64	3	This indicates the presence of fluid collection in the air sacs	Pathological condition
Consolidation	FLOAT64	3	This indicates when the air within small airways of the lungs is replaced with a fluid, solid, or other material (such as pus, blood, water, etc.).	Pathological condition

Pneumonia	FLOAT64	3	This indicates the presence of a lung infection causing inflammation and fluid buildup in the lungs.	Pathological condition
Atelectasis	FLOAT64	3	This indicates the presence of the collapse of a lung (or part of a lung) which occurs when the alveoli lose air and deflate.	Pathological condition
Pneumothorax	FLOAT64	3	This indicates the presence of air leakage from the lungs into the space between the chest wall and the lungs. This can cause the lungs to collapse.	Pathological condition
Pleural Effusion	FLOAT64	3	This indicates the presence of a collection of fluid in the space between the chest wall and the lungs	Pathological condition
Pleural Other	FLOAT64	3	This indicates the presence of some condition affecting the pleura (The membrane lining the chest wall and lungs)	Pathological condition
Fracture	FLOAT64	3	This indicates the presence of a break in a bone.	Pathological condition
Support Devices	FLOAT64	3	This indicates the presence of support devices in the image.	Pathological condition
No Finding	FLOAT64	3	This indicates the absence of all above pathologies	Pathological condition

Breakdown of Pathological Condition Values

For each training instance and pathological condition, a value indicates the presence of (or lack thereof) the specified condition. These values were extracted using the CheXbert labeler and are provided in the blob:

- 1: The condition is present
- 0: The condition is not present
- -1: The labeler was uncertain if this condition was present
    - For our analysis, we map these values to 2
- N/A: The condition was not mentioned in radiological reports
    - For our analysis, we map these values to 0